

NodeJs Backend Developer Interview Preparation Guide

Node.js | TypeScript | NestJS | AWS | REST APIs | Databases
Microservices | Security | System Design | DevOps

This guide explains every topic from the interview question bank in a question-and-answer format designed for quick revision before an interview. Each answer is concise but complete, covering the concept an interviewer expects you to articulate, along with practical examples where relevant.

Topics Covered

1. Basic Node.js	8. API Design & Scalability
2. Basic TypeScript	9. Security
3. Basic AWS	10. Cloud & DevOps
4. Basic REST API	11. Performance & Optimization
5. Database	12. Project Related Questions
6. Microservices	13. System Design
7. NestJS	14. Advanced Backend Topics

How to use this guide

- Each section groups related questions exactly as they appear in the question bank, in the same order.
- Answers are written for a 1-2 minute spoken reply — the depth an interviewer expects, not a textbook chapter.
- For 'difference' questions, lead with the one-line distinction, then the detail; interviewers reward crisp framing.
- Code snippets show the minimal working pattern; adapt the names and structure to your real project.
- For project-related questions (Section 12), replace the generic guidance with your own concrete examples and numbers.

1. Basic Node.js

Q: What is Node.js and why is it single-threaded?

A: Node.js is a JavaScript runtime built on Chrome's V8 engine that lets you run JavaScript outside the browser, primarily for server-side applications. It is single-threaded by design: it uses an event loop to handle many concurrent connections on one main thread instead of spawning a new OS thread per request. This avoids the memory and context-switching overhead of thread-per-connection models and keeps I/O cheap. For CPU-bound work Node offloads to worker threads or the libuv thread pool (default 4 threads), so the main loop never blocks.

Q: What is the difference between require and import?

A: require is CommonJS (Node's original module system) and is synchronous, loaded at runtime, and can be called conditionally. import is ES Modules (ESM), statically analyzed at compile time, supports tree-shaking, and must be at the top level. CommonJS exports via module.exports; ESM uses export / export default. ESM is the modern standard; enable it with "type": "module" in package.json.

Q: What is package.json and package-lock.json?

A: package.json is the manifest of a Node project: it declares dependencies, scripts, metadata (name, version), and config. package-lock.json records the exact resolved version and integrity hash of every dependency (including transitive ones) so that every install across machines produces an identical node_modules tree. Commit the lock file; never edit it by hand.

Q: What are callbacks? What problem do Promises solve over callbacks?

A: A callback is a function passed as an argument to be invoked later when an async operation completes. Nesting many callbacks leads to 'callback hell' (deeply nested, hard-to-read code) and makes error handling error-prone. Promises solve this by flattening the chain with .then()/.catch(), guaranteeing a single resolution, and propagating errors through the chain automatically.

Q: What is async/await and how does it relate to Promises?

A: async/await is syntactic sugar over Promises that lets you write asynchronous code that reads like synchronous code. An async function always returns a Promise; await pauses execution inside the async function until the awaited Promise settles. Errors are handled with try/catch. Under the hood every await is a .then(), so it is the same mechanism with cleaner control flow.

Q: What is middleware in Express.js?

A: Middleware is a function with the signature (req, res, next) that runs during the request-response cycle. It can inspect/modify the request and response, end the cycle (by sending a response), or call next() to pass control to the next middleware or route handler. Use cases: logging, auth, body parsing, CORS, error handling. Order matters — middleware registered first runs first.

Q: What is the difference between let, const, and var?

A: var is function-scoped and hoisted with a default of undefined; it can be redeclared. let is block-scoped and not accessible before its declaration (temporal dead zone). const is also block-scoped but cannot be reassigned (though objects/ arrays declared const are still mutable). Prefer const by default, let when reassignment is needed, and avoid var entirely in modern code.

Q: What is npm vs npx?

A: npm (Node Package Manager) installs packages from the registry and manages dependencies. npx (Node Package eXecute) executes a package binary without permanently installing it — useful for running CLIs like create-react-app or prisma once, always using the latest version. npx will install the package temporarily if it is not already present.

Q: What are environment variables and how do you use them in Node.js (dotenv)?

A: Environment variables are key-value settings loaded into process.env that let you externalize configuration (DB URLs, API keys, ports, secrets) per environment without hardcoding. The dotenv package reads a .env file (git-ignored) and loads its values into process.env at startup.

```
require('dotenv').config(); const dbUrl = process.env.DATABASE_URL;
```

Q: What is the difference between synchronous and asynchronous code?

A: Synchronous code runs line by line and blocks the thread until each statement finishes — fine for fast CPU work but disastrous for I/O. Asynchronous code starts an operation and continues immediately, processing the result later via a callback, Promise, or event. Node's event loop is built around non-blocking async I/O so one thread can serve thousands of connections.

Q: What are Streams in Node.js?

A: Streams are objects that let you read or write data chunk by chunk instead of loading the whole thing into memory. There are four types: Readable, Writable, Duplex (both), and Transform (modify in-flight). Streams are ideal for large files, HTTP responses, or video/audio — they keep memory low and start emitting data before the full payload arrives. pipe() chains a readable into a writable.

Q: What is CORS?

A: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that lets a server declare which origins may access its resources. When a browser makes a cross-origin request, the server must return the Access-Control-Allow-Origin header (and others) permitting it; otherwise the browser blocks the response. In Express you enable it with the cors middleware. Preflight OPTIONS requests are sent for non-simple requests to verify permissions before the real request.

Q: What is Buffer in Node.js?

A: A Buffer is a fixed-size chunk of raw binary memory allocated outside the V8 heap. It represents a sequence of bytes and is the standard way to handle binary data — file I/O, TCP streams, image processing, cryptography. Buffers are to binary what strings are to text. Create one with Buffer.from('hello') or allocate with Buffer.alloc(size).

Q: Explain the Event Loop.

A: The event loop is the mechanism that lets Node perform non-blocking I/O on a single thread. It continuously cycles through phases: timers (setTimeout/setInterval callbacks), pending callbacks, poll (retrieves new I/O events), check (setImmediate), and close callbacks. Microtasks (Promise resolutions, process.nextTick) run after each phase, before moving to the next. When the loop has nothing left to do, the process exits. I/O is offloaded to libuv's thread pool, and results are queued back to the loop.

Q: Explain Callbacks with an example.

A: A callback is a function passed to an async operation that gets invoked when the operation completes. Below, fs.readFile reads the file and then calls the callback with either an error or the data.

```
const fs = require('fs'); fs.readFile('data.txt', 'utf8', (err, data) => { if (err) return console.error(err); console.log(data); });
```

Q: Error handling in Node.js.

A: Use three layers: (1) try/catch for sync code and around awaited Promises; (2) .catch() on Promise chains; (3) a centralized error-handling middleware (err, req, res, next) in Express that logs and sends a consistent error response. Always listen for 'uncaughtException' and 'unhandledRejection' at the process level to log and gracefully restart — an uncaught exception leaves the process in an undefined state. Never swallow errors silently.

Q: Common challenges faced in your current project.

A: Structure this answer around 2-3 real challenges: e.g., handling high-throughput APIs by adding Redis caching and connection pooling; debugging a memory leak with heap snapshots in production; coordinating a zero-downtime database migration. For each, state the problem, the root cause you identified, the fix you shipped, and the measurable outcome (latency dropped X%, errors down Y%).

2. Basic TypeScript

Q: What is TypeScript and how is it different from JavaScript?

A: TypeScript is a statically-typed superset of JavaScript that compiles to plain JavaScript. It adds optional types, interfaces, generics, and compile-time error checking, catching bugs before runtime. JavaScript is dynamically typed — type errors surface only when the code runs. TS improves refactoring safety, editor autocompletion, and team scalability, but it ships as JS after compilation.

Q: What are the basic types in TypeScript?

A: The primitives are string, number, and boolean. Beyond them: any (opt out of checking), unknown (a type-safe counterpart of any — you must narrow it before use), never (a value that never occurs, e.g. a function that always throws or an exhaustive switch), arrays (type[]), tuples, enum, void, null, and undefined. Use unknown over any whenever possible.

Q: What is an Interface? Give a simple example.

A: An interface defines the shape an object must have — a contract that the compiler enforces. It is structural (duck-typed) and erased at runtime. It is the standard way to type API payloads, DTOs, and configuration objects.

```
interface User { id: number; name: string; email?: string; // optional } const u: User = { id: 1, name: 'Asha' };
```

Q: What is the difference between any and unknown?

A: Both accept any value, but any disables all type checking — you can call anything on it without error. unknown is type-safe: you cannot perform operations on it until you narrow it (via typeof, instanceof, or a type guard). Prefer unknown for values whose type you do not yet know, because it forces you to verify before use.

Q: What are optional (?) and readonly properties?

A: The ? suffix marks a property optional — it may be omitted when creating the object. readonly marks a property immutable after initialization — it can be set only in the constructor or at object creation, never reassigned later. Both apply to interfaces, types, and classes.

```
interface Config { readonly apiKey: string; timeout?: number; }
```

Q: What is a TypeScript Enum?

A: An enum is a way to give friendly names to a set of numeric or string constants, making code more readable and self-documenting. Numeric enums auto-increment from the first value (default 0). String enums are recommended for debugging because they serialize to meaningful strings.

```
enum Role { Admin = 'ADMIN', User = 'USER', Guest = 'GUEST' } const r: Role = Role.User;
```

Q: How do you compile TypeScript into JavaScript (tsc)?

A: The TypeScript compiler (tsc) reads .ts files and emits .js. Run `tsc file.ts` for a one-off compile, or use a `tsconfig.json` to configure the whole project (target, module, outDir, strict). In a NestJS app this is handled by the build toolchain automatically.

```
tsc --init # create tsconfig.json tsc # compile per config npx tsc --noEmit # type-check without emitting
```

3. Basic AWS

Q: What is AWS Lambda? What does "Serverless" mean?

A: Lambda is a compute service that runs your code in response to events (HTTP requests, S3 uploads, SQS messages, schedules) without you managing any server. You upload the function and AWS provisions, scales, and bills only for execution time (ms granularity). 'Serverless' means the cloud provider owns provisioning, scaling, and OS maintenance — you focus on code. Cold starts (the first invocation after idle) add latency; tune memory and keep dependencies small to reduce them.

Q: Difference between EC2 and Lambda.

A: EC2 is a virtual server you provision and manage — you pick the instance type, OS, and scaling; you pay while it runs and you handle patching and autoscaling. Lambda is event-driven and fully managed — no server to patch, scales automatically, billed per invocation. Use EC2 for long-running, stateful, or tightly-controlled workloads; Lambda for short, event-triggered functions and bursty traffic.

Q: What is Amazon S3 used for?

A: S3 (Simple Storage Service) is object storage for files of any size — images, videos, backups, logs, static websites, data lakes. You store objects in buckets, each with a unique key. It offers 99.999999999% (11 nines) durability, versioning, lifecycle rules (auto-archive to Glacier), and access control. Common backend use: storing user-uploaded files and serving them via presigned URLs.

Q: What is API Gateway used for?

A: API Gateway is a managed entry point that exposes HTTP/WebSocket APIs and routes requests to backends — Lambda, EC2, ECS, or other AWS services. It handles throttling, authentication (Cognito, custom authorizers), request/response transformation, CORS, and usage plans. It decouples clients from backend internals and is the standard front door for serverless architectures.

Q: Difference between ECS and ECR.

A: ECS (Elastic Container Service) is a container orchestration service that runs and manages Docker containers on clusters of EC2 instances or on Fargate (serverless). ECR (Elastic Container Registry) is a managed Docker registry where you store, version, and pull container images. You build an image, push it to ECR, then ECS pulls and runs it. They work together: ECR stores, ECS executes.

Q: What is CloudWatch used for?

A: CloudWatch is AWS's observability service — it collects logs, metrics, and traces and lets you set alarms and dashboards. Use it to monitor Lambda invocations and errors, EC2 CPU, API Gateway latency, S3 access, and application logs via the CloudWatch Agent or structured logging. Alarms can trigger autoscaling, SNS notifications, or Lambda functions automatically.

Q: IAM Role vs IAM User.

A: An IAM User is an identity for a person or application that needs long-lived credentials (access key + secret). An IAM Role is an identity with a set of permissions that is assumed temporarily — it uses short-lived STS tokens. Roles are the recommended way to grant permissions to AWS services (e.g., a Lambda that needs to read S3) and for cross-account access. Prefer roles over long-lived user keys wherever possible.

Q: How do you use CloudWatch in real projects?

A: Wire every service to emit structured JSON logs (correlation ID, level, message) to CloudWatch Logs. Create metric filters or use embedded metric format to derive business metrics. Set alarms on error rates, latency (p99), and throttling, wired to SNS/Slack for paging. Build dashboards per service for a live operational view. Use Logs Insights to query across log groups when debugging incidents.

Q: How do you make an S3 bucket private?

A: By default a new bucket is private — block all public access (the 'Block Public Access' settings), keep the bucket policy and ACLs closed, and enable server-side encryption (SSE-S3 or SSE-KMS). For controlled access, use a bucket policy that allows actions only from a specific IAM role or VPC endpoint, and give end users short-lived presigned URLs rather than public read.

4. Basic REST API

Q: What is REST?

A: REST (Representational State Transfer) is an architectural style for designing networked APIs over HTTP. It models resources as URLs (e.g. /users/42) and uses standard HTTP methods as verbs to act on them. REST is stateless — each request carries everything the server needs. Responses are typically JSON.

Q: What makes an API RESTful?

A: Stateless communication; resource-based URLs (/orders/5 not /getOrder?id=5); use of standard HTTP methods (GET/POST/PUT/PATCH/DELETE) with their intended semantics; correct status codes; JSON as the media type; HATEOAS links where appropriate; and a uniform interface. Consistent naming and versioning (/v1/...) round out a clean REST design.

Q: Common HTTP methods.

A: GET retrieves a resource and must be safe and cacheable. POST creates a new resource (non-idempotent). PUT replaces a resource entirely at a known URL (idempotent). PATCH partially updates a resource (not strictly idempotent). DELETE removes a resource (idempotent). GET and DELETE should never change server state beyond the obvious; POST does.

Q: Common HTTP status codes.

A: 200 OK (success), 201 Created (new resource made), 204 No Content (success, no body). 400 Bad Request (malformed input), 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but not allowed), 404 Not Found, 409 Conflict (duplicate). 500 Internal Server Error (server fault), 502/503/504 (gateway / service unavailable / timeout). Use the most specific code.

Q: Difference between PUT and PATCH.

A: PUT replaces the entire resource — the client sends the full new representation; omitted fields are typically cleared. It is idempotent: repeating the same PUT yields the same end state. PATCH sends only the fields to change (a partial delta), so it is more bandwidth-efficient but not always idempotent (depends on the patch format).

Q: What is JSON and why is it used in APIs?

A: JSON (JavaScript Object Notation) is a lightweight, human-readable, language-agnostic data-interchange format built on key-value pairs and arrays. It is used in APIs because it is compact, easy for both humans and machines to parse, natively supported by virtually every language, and maps cleanly to most programming-language objects. It replaced XML for most web APIs.

Q: API Idempotency.

A: Idempotency means calling an endpoint N times has the same effect as calling it once. GET, PUT, and DELETE are idempotent by design; POST is not. For non-idempotent operations like payments, attach a client-generated idempotency key (header or body) so the server can detect and dedupe retries, returning the original result instead of charging twice.

Q: How do you optimize APIs?

A: Add caching (Redis or HTTP Cache-Control/ETag) to cut DB load; use pagination (offset/limit or cursor) so responses stay small; compress responses with gzip/brotli; select only needed fields (sparse fieldsets); use connection pooling; run expensive work asynchronously via a queue; index the database for the queries the API issues; and monitor p95/p99 latency to find bottlenecks.

5. Database

Q: Primary Key vs Foreign Key.

A: A primary key uniquely identifies each row in a table — it must be non-null and unique, and there is only one per table. A foreign key is a column (or set) that references the primary key of another table, enforcing referential integrity — you cannot insert a row whose foreign key has no matching parent, and cascading rules define what happens when the parent is deleted or updated.

Q: SQL vs NoSQL.

A: SQL databases (PostgreSQL, MySQL) are relational, schema-fixed, use tables with rows/columns, and guarantee ACID transactions — best for structured data with complex joins and strong consistency. NoSQL databases (MongoDB, DynamoDB, Redis) are non-relational, schema-flexible, horizontally scalable, and favor eventual consistency — best for unstructured/semi-structured data, high write throughput, and rapid schema evolution. Choose by the workload, not by fashion.

Q: What is a JOIN?

A: A JOIN combines rows from two or more tables based on a related column (usually a foreign key). INNER JOIN returns only rows that match in both tables. LEFT JOIN returns all rows from the left table plus matches from the right (NULL where none match). RIGHT JOIN is the mirror of LEFT. FULL OUTER JOIN returns all rows from both, matched where possible.

Q: What is Normalization?

A: Normalization is the process of organizing tables and columns to reduce data redundancy and improve integrity. You split data into related tables and use foreign keys to link them. The common forms: 1NF (atomic values), 2NF (no partial dependency on a composite key), 3NF (no transitive dependency). Over-normalization can hurt read performance — denormalize selectively for hot read paths.

Q: What is an Index and why does it improve performance?

A: An index is an auxiliary data structure (usually a B-tree) on one or more columns that lets the database find rows without scanning the whole table — turning an $O(n)$ scan into roughly $O(\log n)$. It dramatically speeds up WHERE, JOIN, and ORDER BY operations on large tables. Trade-off: indexes add write overhead and storage. Index the columns you filter and join on, but avoid over-indexing write-heavy tables.

Q: DELETE vs TRUNCATE vs DROP.

A: DELETE removes rows one by one (optionally with a WHERE), fires triggers, can be rolled back, and is slow on large tables. TRUNCATE empties the whole table in one fast operation — it deallocates data pages, resets auto-increment, and is minimally logged, so it cannot be row-filtered. DROP removes the entire table structure and its data — the table ceases to exist.

Q: Query Tuning.

A: Read the EXPLAIN plan; look for sequential scans on large tables, missing indexes, or expensive sorts. Add indexes on filter/join columns; rewrite queries to avoid N+1 patterns (use joins or batching); select only needed columns instead of SELECT *; avoid functions on indexed columns in WHERE (they disable the index); use LIMIT for pagination; and rewrite correlated subqueries as joins. Measure before and after.

Q: Database Migration.

A: A migration is a versioned, script-based change to the database schema (create table, add column, etc.) tracked in source control so schema evolves with the code. Tools like TypeORM migrations, Prisma migrate, or Flyway generate up/down scripts. In production, prefer expand-then-contract: deploy the new (backward-compatible) schema, release the code, then clean up the old columns — this enables zero-downtime migrations.

6. Microservices

Q: What is a Microservice?

A: A microservice is a small, independently deployable service that owns one business capability and its own data store. Services communicate over a network (HTTP/gRPC or async via a message broker). Each can be built, scaled, and deployed independently, allowing teams to move in parallel. The trade-off is operational complexity — distributed transactions, network failures, and observability all become your problem.

Q: Difference between Monolith and Microservices.

A: A monolith packages all modules into one deployable unit — simpler to build, test, and deploy, but scaling forces you to scale everything together and a single bug can bring down the whole app. Microservices split this into independent services that scale and deploy separately, with clearer team boundaries — at the cost of distributed-systems complexity, network latency, and harder debugging.

Q: Role of API Gateway.

A: The API Gateway is the single entry point for all clients. It routes requests to the right microservice, handles authentication, rate limiting, request/response transformation, CORS, and aggregating results from multiple services so the client makes one call. It decouples clients from the internal service topology and lets you evolve backends without touching clients.

Q: Synchronous vs Asynchronous communication.

A: Synchronous (HTTP/gRPC): the caller waits for a response in real time — simple, immediate feedback, but tightly couples availability (if the callee is down, the caller fails) and limits scalability. Asynchronous (message broker — Kafka, RabbitMQ, SQS): the caller publishes an event and continues; the consumer processes it later — decoupled, resilient to downstream outages, and buffers load spikes, but adds eventual consistency and operational complexity.

Q: How do you secure Service-to-Service communication?

A: Use mTLS (mutual TLS) so both sides verify each other's certificates; issue short-lived service tokens (e.g., JWTs signed by a central auth service) so one service can authenticate to another; restrict service access at the network level with VPCs/security groups; use a service mesh (Istio, Linkerd) to enforce policy centrally; and pass a correlation ID on every call for traceable audit logs.

Q: Correlation ID.

A: A correlation ID (or trace ID) is a unique identifier generated at the request's entry point and propagated to every downstream service and log line. It lets you reconstruct the full path of a request across services — invaluable for debugging distributed failures. Generate it at the API Gateway (or in the first service), put it in an HTTP header (e.g., x-correlation-id), and log it everywhere.

7. NestJS

Q: Difference between Express.js and NestJS.

A: Express is a minimal, unopinionated web framework — just middleware and routing; you decide every architectural pattern. NestJS is an opinionated framework built on top of Express (or Fastify) that brings Angular-style structure: modules, controllers, providers, dependency injection, decorators, and built-in support for validation, OpenAPI, microservices, and testing. Use Express for small APIs; use NestJS for large, team-based projects where consistency matters.

Q: What are Modules, Controllers, and Providers in NestJS?

A: A Module (@Module) groups related controllers, providers, and imports into a cohesive feature unit — the building block of a Nest app. A Controller (@Controller) handles incoming HTTP requests and returns responses, mapping routes to handler methods. A Provider (@Injectable) is any class that can be injected as a dependency — services, repositories, factories. Modules wire providers and controllers together and export what other modules may import.

Q: What is Dependency Injection (DI) and how does it work?

A: DI is a pattern where a class receives its dependencies (other objects it needs) from an external container instead of constructing them itself. In NestJS you declare a dependency via the constructor, and the framework's IoC container resolves and injects the right instance. This makes classes loosely coupled, easy to test (inject mocks), and centrally managed. Nest uses TypeScript's reflect-metadata to know what to inject based on the parameter types.

Q: What are DTOs, and why are they used?

A: A DTO (Data Transfer Object) is a plain class that defines the shape of data crossing a boundary — typically the request or response body. NestJS uses DTOs with class-validator decorators (@IsString, @IsEmail, @MinLength) and ValidationPipe to validate incoming data automatically, returning 400 on invalid input. DTOs also document the API contract (via Swagger) and decouple the wire format from internal entities.

Q: How do you implement JWT Authentication in NestJS?

A: Use @nestjs/jwt to sign and verify tokens. Flow: a login route validates credentials, calls JwtService.sign(payload) to issue a token, and returns it. A JwtAuthGuard (or strategy with @nestjs/passport) extracts the token from the Authorization header, verifies it, and attaches the decoded user to the request. Protect routes with @UseGuards(JwtAuthGuard). Store secrets in env vars, never in code.

Q: How do you implement RBAC (Role-Based Access Control)?

A: Store a role per user (e.g., admin, user). Create a custom RolesGuard that reads the user's role from the request (after JWT auth) and compares it to roles declared on the route via a @Roles('admin') decorator. Apply both JwtAuthGuard and RolesGuard with @UseGuards. The RolesGuard returns 403 if the user lacks the required role. For fine-grained control, add permission-based checks (e.g., permissions array) instead of hard-coding roles.

Q: How do you validate request data using ValidationPipe and class-validator?

A: Define a DTO class whose properties are decorated with class-validator rules (@IsString, @IsInt, @IsEmail, @Min). Enable ValidationPipe globally with app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true })) so every incoming body/query/param is validated automatically — invalid payloads get a 400 response listing the errors. whitelist strips unknown properties; transform casts payloads into DTO instances.

Q: How do you handle exceptions in NestJS?

A: Throw a built-in HttpException (or a subclass like BadRequestException, NotFoundException) from anywhere; Nest converts it into the matching HTTP status and JSON response. For app-wide handling, implement a custom ExceptionFilter (@Catch()) registered globally to log errors and return a consistent error shape. Always log the full stack server-side but return a sanitized message to the client.

Q: How do you implement file uploads using Multer?

A: Use @nestjs/platform-express with Multer. The @UploadedFile() decorator on a controller method receives the file; configure storage (disk or memory) and limits (file size, accepted MIME types) via MulterModule or an options factory. Validate the file type and size, then upload to S3 (or your storage) and persist the URL. Below is the minimal pattern.

```
@Post('upload') @UseInterceptors(FileInterceptor('file', { storage: diskStorage({
  destination: './uploads', filename: (req, file, cb) => cb(null, Date.now() + '-' +
  file.originalname), })), }) uploadFile(@UploadedFile() file: Express.Multer.File) { return
{ url: '/uploads/' + file.filename }; }
```

Q: NestJS API request end-to-end flow.

A: Client sends an HTTP request -> Nest routes it to the matching Controller method. Pipes validate/transform the incoming data (DTO). Guards check authentication and authorization (JWT, roles). Interceptors run before and after the handler (logging, caching, response mapping). The Controller calls a Service (provider) that contains business logic and talks to the database (via a repository/ORM). The Service returns data; Interceptors post-process it; if an error is thrown, the `ExceptionHandler` catches it and shapes the error response. The Controller returns the final response to the client.

Q: How will you resolve circular dependency?

A: Two services importing each other create a circular dependency. Resolve it with `forwardRef()`: mark one side with `@Inject(forwardRef(() => OtherService))` so Nest defers resolution. Better still, refactor — extract the shared logic into a third service that both depend on, or emit an event through an `EventEmitter/EventBus` instead of direct coupling. Circular imports are usually a sign of a design smell worth removing.

Q: Difference between Middleware, Guard, Interceptor, Filter, Pipe.

A: Middleware runs first, before route handlers — logging, body parsing. Guards run next and decide if a request is allowed (auth, RBAC) — they return boolean or throw. Pipes run after guards and transform/validate the incoming data (DTO validation, type casting). Interceptors wrap the handler — they run logic before and after the method (logging, caching, response mapping). Exception Filters catch unhandled errors and shape the error response. Order: Middleware -> Guard -> Interceptor (before) -> Pipe -> Handler -> Interceptor (after); Filters catch thrown errors.

8. API Design & Scalability

Q: How do you design a scalable API?

A: Keep the API stateless so any instance can serve any request (enables horizontal scaling). Put a load balancer in front of multiple instances. Cache hot reads in Redis. Use connection pooling to the database and paginate large result sets. Move slow work (emails, file processing) to background queues. Scale the database with read replicas and index wisely. Add rate limiting and circuit breakers. Monitor p95/p99 latency and error rates and scale out when they degrade.

Q: How do you design Rate Limiting?

A: Limit how many requests a client may make per time window to protect the service from abuse and noisy neighbors. Common algorithms: fixed window (counter per window), sliding window (weighted recent), and token bucket (refill rate + burst capacity). Track counts in Redis keyed by client/API key. Return 429 Too Many Requests with `Retry-After` when exceeded. Apply stricter limits on expensive endpoints and at the API Gateway so it rejects before work reaches your service.

Q: API Idempotency.

A: Idempotency guarantees that repeating the same request produces the same result. GET/PUT/DELETE are idempotent; POST is not. For non-idempotent operations like payments, require an idempotency key (UUID from the client) — the server stores the first result keyed by it and returns the same result on retries, preventing duplicate side effects from network retries.

Q: Horizontal Scaling.

A: Horizontal (scale-out) means adding more instances of an application behind a load balancer, versus vertical (scale-up) which means a bigger machine. Horizontal scales to very high traffic with no single point of failure, but requires stateless services (session state externalized to Redis/DB) so any instance can serve any request. It is the standard approach for cloud-native apps.

Q: Stateless APIs.

A: A stateless API stores no session data on the server — every request carries everything it needs (auth token in a header, needed params). Any instance can handle any request, so you can add or remove instances freely. Session/auth state lives in a shared store (JWT, Redis) rather than server memory. This is a precondition for horizontal scaling.

Q: Load Balancer.

A: A load balancer distributes incoming requests across multiple backend instances to spread load and provide redundancy. Algorithms: round-robin, least-connections, IP-hash (sticky). It performs health checks and stops sending traffic to unhealthy instances. Examples: AWS ALB, Nginx, HAProxy. It is the front door that makes horizontal scaling work and hides instance failures from clients.

Q: Redis Caching.

A: Redis is an in-memory key-value store used as a cache to offload the database. The service checks Redis before querying the DB; on a hit it returns immediately, on a miss it queries the DB and writes the result back to Redis with a TTL. Use it for read-heavy, rarely-changing data (config, product catalog, user profile). Set realistic TTLs and invalidate on writes to avoid stale data. Watch cache stampede — mitigate with locking or jitter.

Q: Pagination.

A: Pagination returns results in pages (e.g., `?page=2&limit=20`) so responses stay small and fast, avoiding loading huge datasets. Two styles: offset/limit (simple but slow on deep pages — OFFSET scans skipped rows) and cursor-based (use a cursor like a timestamp/id to fetch the next page; stable under inserts, preferred for large or real-time feeds). Always return metadata (total, next cursor).

Q: Background Jobs.

A: Move slow or non-critical work (sending emails, generating reports, processing uploads) out of the request path into a background queue (BullMQ, SQS, RabbitMQ). The API enqueues a job and returns 202 immediately; a worker consumes it and runs the work. This keeps API responses fast, decouples producers from consumers, retries failures automatically, and smooths load spikes.

Q: Versioning.

A: Versioning lets you evolve an API without breaking existing clients. Strategies: URI versioning (`/v1/users`), header versioning (`Accept: application/vnd.api+json; version=2`), or query param (`?version=2`). URI is the most visible and common. Version from day one, deprecate old versions with a clear timeline, and keep versions parallel until clients migrate.

Q: Monitoring.

A: Track four signal types: metrics (request rate, latency p50/p95/p99, error rate, saturation), logs (structured JSON with correlation ID), traces (distributed across services), and alerts (paged when thresholds breach). Build dashboards per service. Define SLIs/SLOs (e.g., 99.9% of requests under 200ms) and alert on burn rate, not raw values. Tools: CloudWatch, Prometheus, Grafana, Datadog, OpenTelemetry.

Q: API Gateway.

A: The API Gateway centralizes cross-cutting concerns — routing, auth, rate limiting, CORS, request/response transformation, logging, and aggregating calls across microservices. Clients call one stable URL; the gateway routes to internal services. This decouples clients from internal topology, lets backends change independently, and provides a single place to enforce policy. AWS API Gateway, Kong, and Nginx are common choices.

9. Security

Q: How do you secure Service-to-Service communication?

A: Use mTLS so both sides verify certificates; issue short-lived service tokens/JWTs from a central identity provider so services authenticate each other; isolate services in private subnets/VPCs with strict security groups; use a service mesh for centralized policy; rotate keys automatically; and log every call with a correlation ID for audit. Never trust the network — assume zero trust.

Q: JWT Authentication.

A: JWT (JSON Web Token) is a signed token (header.payload.signature) issued after login. The client sends it in the Authorization: Bearer header on every request; the server verifies the signature (HMAC or RSA) to trust the claims without a session lookup — making it stateless and horizontally scalable. Keep tokens short-lived; use refresh tokens for long sessions; store secrets in env vars; never put sensitive data in the payload (it is base64, not encrypted).

Q: OAuth.

A: OAuth 2.0 is an authorization framework that lets a user grant a third-party app access to their data on another service without sharing their password, via access tokens. Flows: Authorization Code (web apps, with PKCE for SPAs), Client Credentials (service-to-service), Refresh Token (renew access). OpenID Connect adds authentication on top. Use it when you need delegated access ("Sign in with Google") rather than building your own identity provider.

Q: HTTPS.

A: HTTPS is HTTP over TLS — it encrypts the connection between client and server so an attacker cannot read or tamper with traffic in transit. TLS provides confidentiality (encryption), integrity (tamper detection via MAC), and authentication (the server proves its identity via a certificate). Always use HTTPS in production, enable HSTS, redirect HTTP to HTTPS, and keep certificates renewed (Let's Encrypt or ACM auto-renewal).

Q: RBAC.

A: Role-Based Access Control assigns each user a role (admin, editor, viewer) and each role a set of permissions; access decisions compare the user's role against the role required by a resource. It simplifies administration — you manage a few roles rather than per-user permissions. For finer control, extend to permission-based (ABAC) checks. Enforce RBAC at both API and UI layers; never rely on client-side hiding alone.

Q: SQL Injection Prevention.

A: SQL injection happens when untrusted input is concatenated into a query, letting an attacker run arbitrary SQL. Prevent it by using parameterized queries / prepared statements (the ORM does this by default), never string-concatenating user input into SQL, validating input with allow-lists, and running the DB with least privilege so even a successful injection is limited. Parameterized queries are the primary defense — they treat input as data, never as code.

Q: Input Validation.

A: Validate every input at the trust boundary (API, message consumer) before use: type, format, length, range, and business rules. Use a schema/DTO with a validator (class-validator, Joi, Zod). Reject early with 400 and a clear error. This prevents injection, overflow, and logic bugs, and is your first line of defense. Never trust client-side validation alone — the server is the authority.

Q: API Gateway Security.

A: The gateway is a natural enforcement point: authenticate every request (JWT, API key, OAuth), apply rate limiting and throttling to prevent abuse, validate request size and content type, enforce CORS and allowed methods, inspect payloads with a WAF (SQLi/XSS signatures), log every call for audit, and use mTLS for service-to-service traffic behind it. Centralizing this keeps policy consistent across services.

Q: Private S3 Buckets.

A: Keep buckets private by default: enable 'Block Public Access', deny public-read in the bucket policy, keep ACLs closed, and enable server-side encryption (SSE-S3 or SSE-KMS). Grant access only to specific IAM roles; give end users short-lived presigned URLs for temporary access to specific objects rather than exposing the bucket publicly. Enable CloudTrail to audit access.

10. Cloud & DevOps

Q: AWS Lambda.

A: A serverless compute service that runs code in response to events. You upload a function, AWS provisions and scales it automatically, and you pay per invocation and execution duration. Ideal for event-driven workloads (API Gateway triggers, S3 events, schedules) and bursty traffic. Watch cold-start latency, the 15-minute execution limit, and package size — keep dependencies lean.

Q: ECS.

A: Elastic Container Service runs Docker containers on a cluster you manage (EC2) or on Fargate (serverless, no EC2 to manage). You define a task definition (image, CPU/memory, ports, env) and a service that keeps the desired number of tasks running. ECS integrates with ALB, CloudWatch, and ECR. Good for long-running containerized services where you want control without Kubernetes complexity.

Q: ECR.

A: Elastic Container Registry is a managed Docker registry. You push images to a private ECR repo; ECS (or EKS, Fargate) pulls them at deploy time. ECR handles encryption, image scanning for vulnerabilities, and lifecycle policies to clean old images. Integration with IAM means ECS tasks can pull using their task role — no hardcoded credentials.

Q: EC2.

A: Elastic Compute Cloud provides resizable virtual servers (instances). You choose instance type (CPU/RAM), AMI (OS image), and storage; you manage patching and scaling (via autoscaling groups). Use EC2 when you need full OS control, long-running stateful workloads, or custom software not suited to containers/Lambda. Pay per second (Linux) with on-demand, reserved, or spot pricing.

Q: API Gateway.

A: A managed front door that exposes HTTP/REST or WebSocket APIs and routes them to backends (Lambda, ECS, EC2). It handles auth, throttling, CORS, request/response transforms, usage plans, and per-key quotas. It integrates with CloudWatch for metrics and logs. Use it to decouple clients from your backend topology and to centralize security policy.

Q: CloudWatch.

A: AWS's observability service: Logs (structured app logs), Metrics (CPU, latency, custom business metrics), Alarms (threshold alerts wired to SNS/autoscaling), and Dashboards. Use Logs Insights to query logs, metric filters to derive metrics from log patterns, and X-Ray for distributed tracing. It is the default monitoring stack for AWS-hosted services.

Q: IAM.

A: Identity and Access Management controls who (principal) can do what (action) on which resource. Principals are Users (people/apps), Roles (assumed temporarily by services or cross-account), and Groups. Policies are JSON documents granting permissions. Follow least privilege — grant only the actions a service needs, use roles for services, rotate user keys, and enable MFA for human users.

Q: S3.

A: Object storage for files of any size — uploads, backups, static sites, logs, data lakes. Buckets hold objects keyed by name. Features: versioning, lifecycle rules (archive to Glacier, expire old objects), encryption (SSE-S3/KMS), and fine-grained access via bucket policies. In backends, store user uploads in S3 and serve via presigned URLs; never store files in the database.

Q: Database Migration (DevOps).

A: Migrate schema in lock-step with code using expand-then-contract: deploy the new backward-compatible schema first, then release code that uses it, then run a contract migration to drop the old columns. Use tools (TypeORM/Prisma migrate, Flyway, Liquibase) and run migrations as a deploy step with rollback scripts. For large data migrations, do them in batches during low-traffic windows and validate row counts before and after. Never run a destructive migration without a backup.

Q: Monitoring & Logging.

A: Emit structured JSON logs (level, timestamp, correlation ID, message, context) to a central store (CloudWatch, ELK). Collect metrics (rate, latency, errors, saturation) and set alerts on SLO burn rate. Use distributed tracing to follow a request across services. Build dashboards per service. Define runbooks for common alerts so on-call engineers can respond quickly. Observability is not optional in production.

11. Performance & Optimization

Q: Query Tuning.

A: Read the EXPLAIN plan; eliminate sequential scans on large tables by adding indexes on filter/join columns; avoid SELECT * (fetch only what you need); rewrite N+1 queries into a single JOIN or a batched IN query; avoid wrapping indexed columns in functions (disables the index); use LIMIT and pagination; replace correlated subqueries with joins. Measure before and after — never tune blind.

Q: Database Indexing.

A: Add B-tree indexes on columns used in WHERE, JOIN, and ORDER BY to turn table scans into lookups. Use composite indexes for multi-column queries (order matters — most selective first). Add a unique index where you enforce uniqueness. Avoid over-indexing write-heavy tables — every index slows inserts/updates. Monitor and drop unused indexes. Consider partial or covering indexes for specific query patterns.

Q: Connection Pooling.

A: Opening a new DB connection per request is expensive (TCP handshake, auth, session). A pool keeps a set of reusable connections and hands them out on demand, amortizing the cost and bounding concurrent connections to the DB. Configure pool size to match the DB's capacity (e.g., 10-50), set idle and acquisition timeouts, and monitor wait time — high wait means the pool is too small or queries are slow.

Q: Redis Caching.

A: Redis offloads reads from the database. Check Redis first; on a hit return immediately; on a miss query the DB, write the result to Redis with a TTL, and return. Use for read-heavy, low-churn data. Invalidate or update on writes; set realistic TTLs to bound staleness. Prevent cache stampede on cold keys with locking or jitter. Monitor hit rate — below ~80% the cache may be hurting more than helping.

Q: Pagination.

A: Return results in pages to keep payloads small. Offset/limit is simple but slows on deep pages (OFFSET scans skipped rows). Cursor-based pagination (fetch rows after the last seen id/timestamp) is stable under inserts and faster for large datasets — preferred for feeds. Always include next-page metadata and use indexes on the sort/cursor column.

Q: Async Processing.

A: Move slow work out of the request path into a background queue. The API enqueues a job and returns immediately (202 Accepted); a worker processes it and updates status. This keeps API responses fast, smooths load spikes, enables retries and scheduling, and decouples producers from consumers. Use BullMQ, SQS, or RabbitMQ.

Q: API Optimization.

A: Cache hot reads (Redis, HTTP ETag); paginate large responses; compress with gzip/brotli; select only needed fields; use connection pooling; batch DB queries to avoid N+1; parallelize independent calls with Promise.all; move slow work to queues; index the DB for the queries the API issues; and monitor p95/p99 to find the slowest endpoints, then optimize those first.

Q: Rate Limiting.

A: Cap requests per client per time window to protect the service. Use a token-bucket or sliding-window algorithm tracked in Redis keyed by client/API key. Return 429 with Retry-After when exceeded. Apply stricter limits on expensive endpoints and enforce at the API Gateway so traffic is rejected before it reaches your service.

Q: Horizontal Scaling.

A: Add more identical instances behind a load balancer to handle more traffic. Requires stateless services (externalize session state to Redis/DB) so any instance can serve any request. Scale out on metrics (CPU, queue depth, latency) via autoscaling. Cheaper per unit and more fault-tolerant than scaling up a single machine — the standard cloud-native approach.

Q: Correlation IDs.

A: Generate a unique ID at the request entry point (API Gateway or first service) and propagate it in an HTTP header (x-correlation-id) to every downstream service and every log line. This lets you trace a single request across services and reconstruct the full call chain — essential for debugging distributed failures and for audit.

12. Project Related Questions

Q: Explain your current project architecture.

A: Describe the high-level shape: the client (web/mobile), the API Gateway, the microservices (or monolith) and their responsibilities, the databases per service, caching (Redis), messaging (SQS/Kafka), and the cloud (AWS). Mention auth flow, how services communicate, and CI/CD. Keep it to 2-3 minutes, leading with the domain and the user-facing flow, then the technical layers. Adapt the depth to the interviewer's cues.

Q: What challenges did you face?

A: Pick 2-3 concrete challenges with measurable impact: a performance bottleneck you diagnosed and fixed (e.g., an N+1 query causing 3s responses), a production incident you debugged using logs/traces, or a migration you executed safely. For each: the problem, root cause, what you did, and the outcome. Avoid vague answers like 'we had some scaling issues' — be specific and quantitative.

Q: How did you optimize performance?

A: Walk through a real optimization: identify the bottleneck (slow query, high p99, DB CPU), measure with profiling/logs, apply the fix (add an index, add Redis cache, paginate, batch queries, move to a queue), and re-measure to show improvement (latency dropped from 2s to 200ms). Interviewers want the before/after numbers and your reasoning, not a list of techniques.

Q: How did you implement authentication?

A: Describe the flow end to end: login validates credentials against the DB, issues a JWT (access token, short TTL) plus a refresh token (longer, stored in Redis/DB), returns both to the client. Subsequent requests carry the access token in the Authorization header; a guard validates the signature and attaches the user. Refresh-token rotation reissues access tokens. Mention RBAC and how you revoked tokens (a blocklist) when needed.

Q: How did you migrate databases?

A: Describe the expand-contract approach: deploy the new schema (add columns, new table) as a backward-compatible change; release code that writes to both old and new; backfill existing data in batches; switch reads to the new schema; then remove the old columns in a later deploy. Run during low-traffic windows, take a backup, and validate row counts. Never run a destructive migration without a rollback plan.

Q: How do you monitor production systems?

A: Emit structured logs and metrics to CloudWatch (or Prometheus/Datadog); track the four golden signals (latency, traffic, errors, saturation) plus business metrics; set alerts on SLO burn rate and page via PagerDuty/SNS; build per-service dashboards; use distributed tracing (X-Ray, OpenTelemetry) with correlation IDs. Define runbooks for each alert so on-call engineers can act fast.

Q: How do you handle errors?

A: Centralized: throw typed exceptions from services; a global exception filter catches them, logs the full stack (with correlation ID), and returns a consistent error envelope to the client. Distinguish client errors (4xx, return the field errors) from server errors (5xx, return a generic message and page). Never expose stack traces to clients. Retry transient failures with backoff; dead-letter permanently failed jobs.

Q: How do you debug production issues?

A: Start with the symptom: check dashboards and alerts for the time window, pull the correlation ID from a failing request and trace it across services via logs, identify the failing component, form a hypothesis, and verify against data (logs/metrics/traces). If a recent deploy caused it, roll back first, then root-cause. Keep customers informed and write a post-mortem afterward with action items to prevent recurrence.

Q: How do you secure communication between microservices?

A: mTLS for transport security; short-lived service-to-service JWTs for authentication; network isolation via VPCs and security groups so only intended services can reach each other; a service mesh for centralized policy; correlation IDs on every call for audit. Rotate keys automatically and follow least privilege — a service should only call the endpoints it actually needs.

13. System Design Questions

Q: Design a Scalable API.

A: Clarify requirements (read/write ratio, latency, traffic). Make services stateless behind a load balancer for horizontal scaling. Cache hot reads in Redis; paginate large responses; use a read replica for the database and connection pooling. Move slow work to async queues. Add rate limiting and circuit breakers. Shard the database when it hits its write limit. Instrument with metrics and alerts, and autoscale on traffic. Capacity-estimate early so you pick the right tier.

Q: Design Rate Limiting.

A: Decide the limit (per user, per IP, per API key) and the algorithm: token bucket (allows bursts up to bucket size, refills at a rate) is the most flexible. Store counters in Redis (atomic INCR with TTL) for a distributed system. Apply limits at the API Gateway so traffic is rejected before reaching services. Return 429 with Retry-After and X-RateLimit headers. Differentiate limits per endpoint — stricter on expensive ones. Consider a sliding-window log for precision at higher memory cost.

Q: Design a URL Shortener.

A: On POST, generate a short code (base62 of a counter or a hash), store the mapping {shortCode -> longUrl, createdAt, userId} in a DB, and return the short URL. On GET /:code, look up the long URL (cache in Redis for hot codes), return 301 (permanent) or 302 (temporary) redirect. Scale reads with caching and DB read replicas. Handle collisions by retrying. Add analytics (click count) via async events to a queue. Estimate capacity: 100M URLs, ~10:1 read:write.

Q: Design a Notification Service.

A: A service that sends notifications across channels (email, SMS, push, in-app). Producers publish a notification event to a message queue (Kafka/SQS); worker consumers pick the channel-specific sender (SES, Twilio, FCM), render the template, and send. Retry with exponential backoff; dead-letter failures. Track delivery status. Deduplicate (idempotency key). Template engine per channel. Rate-limit per user to avoid spam. Scale workers independently per channel load.

Q: Design an Authentication System.

A: Support signup/login, JWT access tokens (short TTL) + refresh tokens (longer, stored hashed in a DB/Redis), password hashing (bcrypt/argon2), optional MFA (TOTP), and OAuth for third-party login. Login validates credentials and issues both tokens; the access token guards APIs; the refresh endpoint rotates tokens. Revoke via a blocklist (Redis) on logout. Store secrets in a vault. Add rate limiting on login to prevent brute force and lockouts after N failures.

Q: Design a File Upload Service.

A: Client requests a presigned S3 URL from the API; the API generates one (with TTL and size/content-type limits) and returns it. The client uploads directly to S3 (bypassing your servers, saving bandwidth). On completion, the client notifies the API (or an S3 event triggers it), which records metadata in the DB and enqueues a processing job (virus scan, thumbnail, transcoding). Serve files via CDN (CloudFront) for reads. Enforce quotas per user.

Q: Design a Payment System.

A: Never store raw card data — use a PCI-compliant gateway (Stripe, Razorpay). Flow: client creates an order; the API calls the gateway to create a payment intent and returns the client secret; the client confirms payment; a webhook from the gateway confirms the final status; the API updates the order. Make every payment idempotent via an idempotency key. Handle webhooks with signature verification and idempotent processing. Reconcile daily against gateway reports. Add retry and dead-letter for failed webhooks.

Q: Design a Chat System.

A: Real-time messaging between users. Use WebSockets (or Server-Sent Events) for live delivery; a connection manager maps userId -> active socket(s). Persist messages in a DB (or a chat-optimized store like Cassandra/DynamoDB) partitioned by conversation. For offline users, push via FCM/APNs. Scale horizontally with a pub/sub layer (Redis Pub/Sub or Kafka) so a message published on one node reaches subscribers on any node. Maintain message ordering per conversation and support delivery/read receipts.

Q: Design a Logging System.

A: Services emit structured JSON logs to a local agent (Fluentd/Filebeat) that ships to a central pipeline (Kafka or Kinesis). An indexer (Elasticsearch / OpenSearch) stores them; a query/UI layer (Kibana, CloudWatch Logs Insights) lets engineers search by service, level, correlation ID, time range. Add retention policies (hot/warm/cold tiers) to control cost. Index correlation IDs for cross-service tracing. Add alerting on log patterns (error spikes).

Q: Design a Search Service.

A: A service that indexes documents and returns ranked results by query. Use a search engine (Elasticsearch, OpenSearch, or Algolia/Typesense for managed). Producers publish document-change events; an indexer consumer updates the search index (or use CDC from the DB). Support full-text search, filters, facets, and ranking. Cache common queries in Redis. Scale by sharding the index and adding replicas for read throughput. Reindex without downtime by aliasing to a new index and swapping.

14. Advanced Backend Topics

Q: Event Loop.

A: The event loop is what makes Node non-blocking on a single thread. It cycles through phases: timers (setTimeout/setInterval callbacks), pending callbacks, poll (new I/O events), check (setImmediate), and close callbacks. After each phase it drains the microtask queue (Promises, process.nextTick). Heavy sync code blocks the loop and stalls everything — offload CPU-bound work to worker threads or break it into chunks with setImmediate.

Q: Streams.

A: Streams process data in chunks instead of buffering the whole payload, keeping memory low and starting output before input finishes. Four kinds: Readable (`fs.createReadStream`), Writable, Duplex (both, e.g., TCP socket), and Transform (gzip, crypto). `pipe()` chains a readable into a writable. Backpressure pauses the producer when the consumer cannot keep up, preventing memory blowup. Use streams for files, HTTP bodies, and any large or continuous data.

Q: Buffers.

A: A Buffer is a fixed-length sequence of raw bytes stored outside the V8 heap — the standard way to handle binary data (file I/O, TCP, crypto, images). Unlike strings, buffers are not encoded text. Create with `Buffer.from(input)` or `Buffer.alloc(size)`. Convert to/from strings with explicit encoding (`buf.toString('utf8')`). Buffers are to binary what strings are to text.

Q: CORS.

A: CORS (Cross-Origin Resource Sharing) is a browser-enforced policy: a web page at origin A can only fetch resources from origin B if B explicitly allows A via the `Access-Control-Allow-Origin` response header. For non-simple requests the browser sends a preflight `OPTIONS` request; the server must respond with the allowed methods, headers, and origin. Configure CORS in Express with the `cors` middleware, whitelisting specific origins rather than `*` for authenticated APIs.

Q: Middleware.

A: Middleware is a function (`req, res, next`) in the request pipeline that can inspect or modify the request/response, end the cycle, or call `next()` to continue. Common uses: logging, auth, body parsing, CORS, compression, error handling. In Express, `app.use()` applies globally; router-level middleware scopes it to specific routes. Order matters — middleware registered first runs first. In NestJS the equivalent layers are Middleware, Guards, Interceptors, Pipes, and Filters.

Q: Error Handling.

A: Layer it: `try/catch` around sync and awaited code; `.catch()` on Promise chains; a centralized Express error middleware (`err, req, res, next`) or a NestJS `ExceptionHandler` that logs (with correlation ID) and returns a consistent error envelope. Distinguish 4xx (client) from 5xx (server). Never expose stack traces to clients. Listen for `'uncaughtException'` and `'unhandledRejection'` to log and gracefully restart. Retry transient failures with backoff; dead-letter permanently failed jobs.

Q: Callback vs Promise vs Async/Await.

A: A callback is a function passed to be called later — nesting leads to callback hell and hard error handling. A Promise represents a future value with a clean `.then/.catch` chain, single resolution, and automatic error propagation. `async/await` is syntactic sugar over Promises: an `async` function returns a Promise, `await` pauses until it settles, and errors are handled with `try/catch`. They are the same mechanism — `await` compiles to `.then()` — but `async/await` reads as linear code. Always prefer `async/await` in modern Node.

Q: SQL vs NoSQL.

A: SQL (relational) enforces a fixed schema and ACID guarantees; best for structured data, joins, and strong consistency. NoSQL (document, key-value, column, graph) offers a flexible schema and horizontal scaling; best for unstructured data, high write throughput, and rapid iteration, usually with eventual consistency. Choose by the workload and consistency needs; many systems use both — a relational DB for transactional core, NoSQL for caches/logs/search.

Q: API Idempotency.

A: Idempotency means calling an endpoint N times has the same effect as once. GET/PUT/DELETE are idempotent by design. POST is not — so for side-effecting POSTs (payments, order creation) require an idempotency key from the client; the server stores the result keyed by it and returns the same result on retry, preventing duplicate effects from network retries. This is essential for reliability in distributed systems.

Q: Correlation IDs.

A: A correlation ID is a unique identifier created at the request entry point and propagated in an HTTP header to every downstream service and every log line, letting you trace one request across a distributed system. Essential for debugging multi-service failures: grep the ID and see the full call path. Generate it at the API Gateway or first service; log it everywhere; include it in async job messages so background work stays traceable too.

Q: Circular Dependencies.

A: Two modules/services importing each other create a cycle. Symptoms: undefined imports at runtime, initialization order bugs. Fix by using forwardRef() (NestJS) or dynamic require to defer resolution. Better: refactor — extract the shared dependency into a third module both can import, or replace the direct call with an event (EventEmitter / message queue) so neither imports the other. Cycles are usually a design smell worth removing.

Q: Query Optimization.

A: Use EXPLAIN to find sequential scans and expensive sorts on large tables. Add indexes on filter/join/sort columns; use composite indexes with the most selective column first. Avoid SELECT * and function-wrapped indexed columns in WHERE. Rewrite N+1 queries into a single JOIN or batched IN. Use LIMIT and cursor pagination. Cache hot results in Redis. Measure before and after — optimize the queries that actually show up in slow-query logs.

Q: Database Migration.

A: A migration is a versioned, source-controlled script that changes the schema, run as part of deploy. Use expand-then-contract for zero downtime: add the new columns/table (backward compatible), release code that uses them, backfill data, switch reads, then remove the old. Always provide a down/rollback script, take a backup, run large migrations in batches during low-traffic windows, and validate row counts. Never run a destructive migration without a rollback plan.